

Содержание

1	Короткая документация по реализации	1
1.1	1. Алгоритм МиниГОСТ	1
1.2	2. Алгоритм расчёта Δ_{pi}	4
1.3	3. Как всё используется в проекте	5
1.4	4. Короткий вывод	7
1.5	5. Эксперимент по восстановлению двух последних раундовых ключей	7

1. Короткая документация по реализации

Ниже описано только то, что подтверждается текущим кодом проекта.

Основные источники: - `src/minigost.c` - `include/minigost.h` - `tools/prepare_phase.py` - `tools/generate_lookup_table.py` - `tools/compute_delta_pi.c` - `Makefile`

1.1. 1. Алгоритм МиниГОСТ

1.1.1. 1.1. Формат данных

По текущему коду: - блок: 16 бит; - полублок: 8 бит; - ключ: 32 бита; - ключ передаётся как `const uint8_t key[4]`; - число раундов: 12.

Публичный API:

```
uint16_t minigost_encrypt_block(uint16_t plaintext, const uint8_t key[4]);
uint16_t minigost_decrypt_block(uint16_t ciphertext, const uint8_t key[4]);
int minigost_encrypt_ecb(const uint8_t *plaintext, uint8_t *ciphertext, size_t length, const uint8_t key[4]);
int minigost_decrypt_ecb(const uint8_t *ciphertext, uint8_t *plaintext, size_t length, const uint8_t key[4]);
```

1.1.2. 1.2. Интерпретация ключа

Ключ понимается так:

$K = (K^{(4)}, K^{(3)}, K^{(2)}, K^{(1)})$

где: - $K^{(4)}$ — старший байт; - $K^{(1)}$ — младший байт.

Соответствие массиву `key[4]`:

```
key[0] =  $K^{(4)}$ 
key[1] =  $K^{(3)}$ 
key[2] =  $K^{(2)}$ 
key[3] =  $K^{(1)}$ 
```

Пример:

```
uint8_t key[4] = {0xA1, 0xB2, 0xC3, 0xD4};
```

означает:

```
 $K^{(4)} = 0xA1$ 
 $K^{(3)} = 0xB2$ 
 $K^{(2)} = 0xC3$ 
 $K^{(1)} = 0xD4$ 
```

1.1.3. 1.3. Как ключ используется в раундах

В коде зашифто расписание:

```
static const uint8_t schedule[12] = {3, 2, 1, 0, 3, 2, 1, 0, 0, 1, 2, 3};
```

Это даёт порядок раундовых ключей:

```
 $K^{(1)}, K^{(2)}, K^{(3)}, K^{(4)},$ 
 $K^{(1)}, K^{(2)}, K^{(3)}, K^{(4)},$ 
 $K^{(4)}, K^{(3)}, K^{(2)}, K^{(1)}$ 
```

То есть по раундам:

```

r1 = K^(1)
r2 = K^(2)
r3 = K^(3)
r4 = K^(4)
r5 = K^(1)
r6 = K^(2)
r7 = K^(3)
r8 = K^(4)
r9 = K^(4)
r10 = K^(3)
r11 = K^(2)
r12 = K^(1)

```

1.1.4. 1.4. Подготовительная функция

В src/minigost.c нет вычисления prep_phase на лету. Вместо этого используется таблица:

```
static const uint8_t PREP_PHASE_TABLE[256] = { ... };
```

Раундовая функция:

```
static inline uint8_t round_function(uint8_t n1, uint8_t k) {
    uint8_t sum = (n1 + k) & 0xFF;
    return PREP_PHASE_TABLE[sum];
}
```

То есть:

$$F(x, k) = \text{PREP_PHASE_TABLE}[(x + k) \bmod 256]$$

1.1.5. 1.5. Как сгенерирован PREP_PHASE_TABLE

Генератор: tools/prep_phase.py.

В нём заданы две 4-битные подстановки:

```

p1 = [12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1]
p0 = [6, 8, 2, 3, 9, 10, 5, 12, 1, 14, 4, 7, 11, 13, 0, 15]

```

Для каждого байта byte от 0 до 255 делается:

```

high_nibble = (byte >> 4) & 0xF
low_nibble = byte & 0xF

```

```

high_sub = p1[high_nibble]
low_sub = p0[low_nibble]

```

```

substituted = (high_sub << 4) | low_sub
result = ((substituted << 5) | (substituted >> 3)) & 0xFF

```

То есть:

```

h = старшие 4 бита byte
l = младшие 4 бита byte

```

```

s(byte) = (p1[h] << 4) | p0[l]
T[byte] = ROTL8(s(byte), 5)

```

Где:

$$\text{ROTL8}(x, 5) = ((x \ll 5) \mid (x \gg 3)) \& 0xFF$$

После этого генератор просто печатает все 256 значений подряд в C-массив.

Пример из кода:

```

byte = 0x93 = 10010011b
h = 9
l = 3
p1[9] = 8
p0[3] = 3
substituted = 0x83
result = ROTL8(0x83, 5) = 0x70

```

То же значение проверяется встроенным тестом в `src/minigost.c`.

1.1.6. 1.6. Один раунд шифрования

Блок делится на два байта:

L = старший байт

R = младший байт

Код раунда:

```
uint8_t n1 = (*block >> 8) & 0xFF;
uint8_t n2 = *block & 0xFF;
uint8_t g = round_function(n1, key_element);
uint8_t new_n1 = n2 ^ g;
*block = ((uint16_t)new_n1 << 8) | n1;
```

Это соответствует преобразованию:

$(L, R) \rightarrow (R + F(L, k), L)$

Здесь + понимается в терминах V8, то есть в коде это побитовое XOR.

Эквивалентная запись, которую реально видно в C-коде:

$(L, R) \rightarrow (R \text{ XOR } F(L, k), L)$

1.1.7. 1.7. Полное шифрование блока

Код:

```
for (int round = 0; round < 12; round++) {
    uint8_t key_element = key[schedule[round]];
    minigost_encrypt_round(&block, key_element);
}
```

```
uint8_t n1 = (block >> 8) & 0xFF;
uint8_t n2 = block & 0xFF;
block = ((uint16_t)n2 << 8) | n1;
```

То есть:

1. Выполнить 12 раундов по расписанию ключей.
2. После 12-го раунда поменять местами левый и правый полублок.

Если обозначить вход как (L_0, R_0) , то каждый раунд:

$(L(i+1), R(i+1)) = (R(i) + F(L(i), \text{round_key}(i)), L(i))$

После последнего раунда:

`ciphertext = swap(L12, R12)`

1.1.8. 1.8. Расшифрование блока

Код:

```
uint8_t n1 = (block >> 8) & 0xFF;
uint8_t n2 = block & 0xFF;
block = ((uint16_t)n2 << 8) | n1;

for (int round = 11; round >= 0; round--) {
    uint8_t key_element = key[schedule[round]];
    minigost_decrypt_round(&block, key_element);
}
```

Обратный раунд:

```
uint8_t n1 = (*block >> 8) & 0xFF;
uint8_t n2 = *block & 0xFF;
uint8_t g = round_function(n2, key_element);
uint8_t new_n2 = n1 ^ g;
*block = ((uint16_t)n2 << 8) | new_n2;
```

То есть:

$$(L, R) \rightarrow (R, F(R, k) + L)$$

И снова + в V8 реализовано в коде как XOR, поэтому эквивалентная запись:

$$(L, R) \rightarrow (R, F(R, k) \oplus L)$$

Порядок действий:

1. Сначала делается обратная финальная перестановка.
2. Затем идут 12 обратных раундов в обратном порядке ключей.

1.1.9. 1.9. ЕСВ режим

Код собирает 16-битный блок из двух байтов в big-endian виде:

```
uint16_t block = ((uint16_t)plaintext[i] << 8) | plaintext[i + 1];
```

Каждый блок шифруется независимо:

$$C_j = \text{Enc}_K(P_j)$$

Ограничение: - длина входа должна быть кратна 2; - иначе функция возвращает -1.

1.2. 2. Алгоритм расчёта delta_pi

1.2.1. 2.1. Что считается функцией pi

В tools/compute_delta_pi.c шифрование не вычисляется на лету. Подключается готовая lookup-таблица:

```
#include "lookup_tables/DEADBEEF_lookup.h"
```

Значит:

$$\text{pi}(x) = \text{MINIGOST_LOOKUP_TABLE}[x]$$

для всех x от 0 до 65535.

1.2.2. 2.2. Пространство перебора

По коду: - $N = 65536 = 2^{16}$; - перебираются все β от 1 до 65535; - для каждого β перебираются все α от 1 до 65535.

Нулевые маски не учитываются.

1.2.3. 2.3. Вектор для фиксированного beta

Код:

```
for (int x = 0; x < 65536; x++) {
    uint16_t y = lookup_table[x];
    f[x] = parity16(beta & y) ? -1 : 1;
}
```

Здесь $\text{parity16}(z)$ — чётность числа единиц в z .

То есть строится вектор:

$$\begin{aligned} f_{\beta}(x) &= 1, \text{ если } \text{parity}(\beta \& \text{pi}(x)) = 0 \\ f_{\beta}(x) &= -1, \text{ если } \text{parity}(\beta \& \text{pi}(x)) = 1 \end{aligned}$$

или короче:

$$f_{\beta}(x) = (-1)^{(\beta \& \text{pi}(x))}$$

1.2.4. 2.4. Walsh-Hadamard transform

Код:

```
for (int len = 1; len < 65536; len <= 1) {
    for (int start = 0; start < 65536; start += (len < 1)) {
        for (int i = 0; i < len; i++) {
            int a = f[start + i];
            int b = f[start + i + len];
```

```

        f[start + i] = a + b;
        f[start + i + len] = a - b;
    }
}

```

После этого `f[alpha]` — коэффициент Уолша для маски `alpha`.

1.2.5. 2.5. Корреляция и `delta_pi`

Код:

```

const float norm = 1.0f / 65536.0f;
float correlation = -(float)f[alpha] * norm;
float abs_correlation = fabsf(correlation);

```

То есть:

```

c(alpha, beta) = -f[alpha] / 65536
delta_pi = max |c(alpha, beta)|

```

При нахождении нового максимума сохраняются: `-alpha_max`; `-beta_max`; `-correlation_value`; `-max_correlation`.

1.2.6. 2.6. Что делает программа пошагово

1. Берёт готовую `lookup`-таблицу `pi`.
2. Для каждого `beta != 0` строит вектор длины 65536.
3. Применяет FWT.
4. Для каждого `alpha != 0` вычисляет корреляцию.
5. Ищет максимальный модуль корреляции.
6. Печатает результат и сохраняет его в `delta_pi_result.txt`.

1.3. 3. Как всё используется в проекте

1.3.1. 3.1. Базовая реализация шифра

Основные файлы: `- src/minigost.c` - `include/minigost.h`

Проверка:

```
make test
```

1.3.2. 3.2. Использование как C-библиотеки

Сборка библиотек:

```

make lib/libminigost.a
make lib/libminigost.dylib # macOS

```

Пример интеграции:

```
make example
```

1.3.3. 3.3. Python-обёртка над C

Запуск:

```
make python
```

1.3.4. 3.4. Генерация `lookup`-таблицы

Таблица строится так:

```

for block in range(0x10000):
    encrypted = cipher.encrypt_block(block, key)
    lookup_table.append(encrypted)

```

То есть материализуется полное отображение:

```
x -> pi(x)
```

Поддерживаются форматы: - .bin - .h - .md - .py

Запуск:

```
python3 tools/generate_lookup_table.py --key 0xDEADBEEF
./generate_lookup.sh
```

1.3.5. 3.5. Расчёт delta_pi

Запуск:

```
./run_delta_pi.sh
```

Что делает скрипт: 1. Проверяет наличие lookup_tables/DEADBEEF_lookup.h. 2. Компилирует tools/compute_delta_pi.c. 3. Запускает build/compute_delta_pi. 4. Перекладывает delta_pi_result.txt в results/.

Важно: tools/compute_delta_pi.c сейчас жёстко подключает конкретный header-файл, так что без смены #include он считает delta_pi только для уже выбранного ключа.

1.3.6. 3.6. Проверка корректности

Полное сравнение Python и C по всем 65536 блокам:

```
make compare
```

Смысл проверки:

```
blocks = range(0x10000)
py_results = run_python_batch(cipher, mode, py_key, blocks)
c_results = run_c_batch(runner, mode, c_key, blocks)
```

1.3.7. 3.7. Минимальный набор рабочих сценариев

```
make test
make example
make benchmark
make compare
make python
python3 tools/generate_lookup_table.py --key 0xDEADBEEF
./generate_lookup.sh
./run_delta_pi.sh
python3 verify_correlation.py
sage verify_in_sage.py
./batch_compute_delta_pi.sh 10
```

1.3.8. 3.8. Эксперимент по восстановлению двух последних раундовых ключей

Основные файлы: - tools/key_recovery_experiment.c - run_key_recovery.sh

Быстрый запуск на одном фиксированном ключе:

```
./run_key_recovery.sh --fixed-key 0xDEADBEEF --top 8
```

Проверочный запуск на двух случайных ключах:

```
./run_key_recovery.sh --count 2 --seed 1 --top 5
```

Полный запуск на 1000 ключах:

```
./run_key_recovery.sh --count 1000 --seed 1 --top 32
```

Рекомендуемый запуск по новому комментарию преподавателя: увеличить материал до 100 / Δ^2 и считать многопоточно:

```
./run_key_recovery.sh --count 1000 --seed 1 --top 32 --threads 10 --m 100
```

Сейчас M = 100 также установлен как значение по умолчанию в самом инструменте. Поэтому эквивалентный запуск:

```
./run_key_recovery.sh --count 1000 --seed 1 --top 32 --threads 10
```

Продолжение уже начатого прогона из существующей папки результатов:

```
./run_key_recovery.sh \
  --resume-dir results/key_recovery/<run_id> \
  --count 1000 \
  --seed 1 \
  --top 32
```

Явное задание числа CPU-потоков:

```
./run_key_recovery.sh --count 1000 --seed 1 --top 32 --threads 10
```

Если нужен полный дамп всех 65536 кандидатов (K'_1 , K'_2) для каждой итерации:

```
./run_key_recovery.sh --count 2 --seed 1 --top 5 --save-full-candidates
```

Что сохраняется: - results/key_recovery/<run_id>/run_config.txt - results/key_recovery/<run_id>/summary.csv
- results/key_recovery/<run_id>/top_candidates_*.csv - results/key_recovery/<run_id>/all_candidates_*.csv
только если включён --save-full-candidates

Как работает продолжение: - читается существующий summary.csv; - определяется число уже завершённых итераций; - генератор ключей проматывается на это число шагов; - новые строки дописываются в тот же summary.csv; - пересчёта уже завершённых итераций не происходит. - выборка открытых текстов для каждой итерации остаётся воспроизводимой, потому что она детерминированно строится из seed, номера итерации и самого ключа.

1.4. 4. Короткий вывод

Сейчас проект состоит из четырёх основных частей:

1. Реализация МиниГОСТ как 12-раундовой сети Фейстеля над 16-битным блоком и 32-битным ключом.
2. Генерация полной lookup-таблицы π : $\{0,1\}^{16} \rightarrow \{0,1\}^{16}$ для фиксированного ключа.
3. Вычисление $\delta_{\pi} = \max |c(\alpha, \beta)|$ через полный перебор масок и FWT.
4. Эксперимент по восстановлению двух последних раундовых ключей ($K^{(1)}$, $K^{(2)}$) по линейному преобладанию 10-раундового префикса.

1.5. 5. Эксперимент по восстановлению двух последних раундовых ключей

Нижне речь уже не про расчёт δ_{π} для полного шифра, а про отдельный эксперимент.

1.5.1. 5.1. Что фиксируется для истинного ключа

Берётся истинный ключ

$K = (K^{(4)}, K^{(3)}, K^{(2)}, K^{(1)})$

и его 10-раундовый префикс

$Fbar_K$ = первые 10 раундов в реальном порядке выполнения:
 $K^{(1)}, K^{(2)}, K^{(3)}, K^{(4)},$
 $K^{(1)}, K^{(2)}, K^{(3)}, K^{(4)},$
 $K^{(4)}, K^{(3)}$

Здесь финальная перестановка T ещё не применяется.

Для $Fbar_K$ полным перебором ищутся маски α, β , на которых достигается

$\delta^{(Fbar_K)} = \max |\delta_{\pi}(\alpha, \beta)^{(Fbar_K)}|$

1.5.2. 5.2. Как достраивается полный шифр

После 10 раундов в полном шифре остаются ещё два раунда с ключами

$K^{(2)}, K^{(1)}$

и финальная перестановка T .

В этом разделе я фиксирую запись в варианте из замечания преподавателя:

$E_K = Fbar_K \circ H_2 \circ H_1$

То есть лишний T из этой формулы убран. В самой реализации снятие последних двух раундов делается не через сокращения с T , а пошагово, чтобы не было двусмысленности.

1.5.3. 5.3. Что проверяется для кандидата K'

Перебирается кандидат

$$K' = (K'_1, K'_2) \text{ in } V_{16}$$

где: - K'_1 — кандидат на $K^{(1)}$; - K'_2 — кандидат на $K^{(2)}$.

Для каждого открытого текста x берётся

$$y = E_{K'}(x)$$

а затем с y частично снимаются финальная перестановка и два последних раунда:

1. $y_0 = T(y)$
2. $y_1 = H^{(-1)}_{K'_1}(y_0)$
3. $z = H^{(-1)}_{K'_2}(y_1)$

Если кандидат верный, то z должно приближать $Fbar_K(x)$.

1.5.4. 5.4. Счётчик и оценка преобладания

Пусть

$$n = \text{ceil}(M / (\delta^{Fbar_K})^2)$$

где в базовой постановке M выбирается как положительная константа.

В текущем эксперименте по замечанию преподавателя используется

$$M = 100.$$

Если величина $100 / (\delta^{Fbar_K})^2$ превосходит 65536, то фактически берётся весь доступный набор открытых текстов, потому что пространство V_{16} содержит только 65536 различных значений.

Для самого эксперимента нужно брать не префикс пространства V_{16} , а случайную выборку без повторений из всего V_{16} :

$$x_1, x_2, \dots, x_n \text{ in } V_{16}$$

равновероятно и без возвращения.

Префиксная последовательность

$$0, 1, 2, \dots, n - 1$$

может давать искажённую оценку линейного преобладания и поэтому для итогового эксперимента не используется.

Для фиксированного кандидата (K'_1, K'_2) считается

$$S = \#\{x : \langle \alpha, x \rangle = \langle \beta, z \rangle\}$$

где: - z получено после частичного снятия двух последних раундов; - $\langle u, v \rangle$ — скалярное произведение над V_{16} , то есть сумма по модулю 2.

Дальше:

$$p_{\text{hat}} = S / n$$

$$\delta_{\text{hat}} = 2 * p_{\text{hat}} - 1$$

Для сравнения кандидатов используется модуль $|\delta_{\text{hat}}|$.

1.5.5. 5.5. Что ожидается

Теоретическое ожидание:

- если $K' = (K^{(1)}, K^{(2)})$, то $|\delta_{\text{hat}}|$ должно быть близко к δ^{Fbar_K} ;
- если K' ложный, то $|\delta_{\text{hat}}|$ должно быть заметно меньше.

Именно это и измеряется в эксперименте: для каждого истинного ключа сохраняются - α , β , δ^{Fbar_K} ; - истинная пара $(K^{(1)}, K^{(2)})$; - лучший найденный кандидат; - место истинной пары в ранжировании по $|\delta_{\text{hat}}|$; - top-кандидаты и их оценки.

1.5.6. 5.6. Что делает реализованный инструмент

В `tools/key_recovery_experiment.c` одна итерация устроена так:

1. Выбирается истинный 32-битный ключ K .
2. Для этого ключа строится lookup-таблица 10-раундового префикса $Fbar_K$.
3. По этой таблице полным перебором находится $\delta^{(Fbar_K)}$ и соответствующие α, β .
4. По формуле $n = \lceil M / \delta^2 \rceil$ выбирается размер выборки открытых текстов.
5. Из пространства V_{16} выбираются n случайных различных открытых текстов.
6. Для каждого выбранного x вычисляется полный шифртекст $y = E_K(x)$.
7. Перебираются все 65536 кандидатов (K'_1, K'_2).
8. Для каждого кандидата считается оценка δ_{hat} .
9. Сохраняются ранг истинной пары, лучший кандидат и top-список.
10. После каждой завершённой итерации строка сразу дописывается в `summary.csv`.

1.5.7. 5.7. Где в коде находится ускорение

Наивный способ для одной итерации выглядел бы как:

```
для каждого кандидата ( $K'_1, K'_2$ )
  для каждого  $x$  из выборки
    частично снять два последних раунда
    проверить линейное соотношение
```

В текущей реализации это ускорено так:

- для каждого K'_1 сначала считаются веса `weights[u]` по промежуточному байту u ;
- затем все 256 вариантов K'_2 добиваются через короткую свёртку по u ;
- поэтому этап восстановления ключа имеет сложность порядка $256 * n + 256 * 256 * 256$, а не $65536 * n$.

Из-за этого на практике почти всё время уходит не на перебор кандидатов, а на поиск $\delta^{(Fbar_K)}$ для 10-раундового префикса.

Дополнительно этот поиск теперь распараллелен по маскам β на CPU:

- каждому потоку выдаётся свой диапазон β ;
- каждый поток независимо строит вектор, делает FWT и ищет локальный максимум;
- затем локальные максимумы сводятся в один глобальный результат.

Поэтому главный источник ускорения сейчас не в стадии восстановления (K'_1, K'_2), а в многопоточном поиске $\delta^{(Fbar_K)}$.

1.5.8. 5.8. Какие поля пишет `summary.csv`

Файл `summary.csv` содержит по одной строке на истинный ключ:

- ITERATION
- KEY
- K4, K3, K2, K1
- ALPHA, BETA
- DELTA_PREFIX
- SAMPLE_COUNT
- TRUE_GUESS_K1, TRUE_GUESS_K2
- TRUE_SCORE, TRUE_BIAS, TRUE_RANK
- BEST_GUESS_K1, BEST_GUESS_K2
- BEST_SCORE, BEST_BIAS
- BEST_TIE_COUNT
- TRUE_IN_BEST_TIES
- DELTA_TIME_SEC
- RECOVERY_TIME_SEC
- TOTAL_TIME_SEC

Это позволяет потом агрегировать: - как часто истинная пара попадает в top-1, top-5, top-10; - распределение TRUE_RANK; - распределение DELTA_PREFIX; - разрыв между истинным и лучшим ложным кандидатом; - среднее время на одну итерацию.

1.5.9. 5.9. Что дают дополнительные CSV-файлы

top_candidates_*.csv сохраняет только лучших кандидатов и подходит для ручного просмотра.

all_candidates_*.csv сохраняет все 65536 кандидатов и нужен, если потом требуется: - строить гистограммы по всем оценкам; - анализировать ложные пики; - считать квантили фона; - отдельно изучать расстояние между истинным ключом и всей остальной массой кандидатов.

1.5.10. 5.10. Что уже измерено на тестовых прогонах

На коротких проверочных запусках получилось:

- 1 итерация, --threads 1: около 7.0 секунд;
- 1 итерация, многопоточно: около 1.3 секунд;
- 2 новые итерации в resume-режиме: около 2.35 секунд суммарно.
- 10 итераций в режиме --m 1000, --threads 10: 11.72 секунд суммарно, и во всех 10 случаях истинная пара оказалась среди кандидатов с максимальным $|\delta_{\hat{h}}|$ (TRUE_RANK = 1, возможно с tie).

Для режима --m 1000, --threads 10 это даёт грубую оценку для полного прогона на 1000 ключах:

порядка 1150–1250 секунд

порядка 19–21 минут

По текущим измерениям: - основное время уходит на поиск $\delta^*(\bar{F}_K)$; - сам перебор всех 65536 кандидатов (K'_1, K'_2) занимает лишь малую долю времени; - resume-режим позволяет безопасно прерывать длинный прогон и потом продолжать его из уже заполненной папки результатов.

1.5.11. 5.11. Замечание о старом прогоне на 1000 ключах

Ранее был выполнен большой прогон:

- каталог: results/key_recovery/20260507_162919_27885
- сводка: results/key_recovery/20260507_162919_27885/summary.csv
- итоговый отчёт: results/key_recovery/20260507_162919_27885/report.md

Однако в том прогоне выборка открытых текстов строилась как префикс

0, 1, 2, ..., n - 1,

а не как случайная выборка без повторений из V_{16} .

После дополнительной проверки выяснилось, что такая префиксная выборка может давать систематически искажённые оценки и ранги. Поэтому этот старый прогон нельзя использовать как окончательный результат восстановления ключа.

Старые файлы оставлены только как исторический артефакт и для сравнения производительности.

Практическое замечание по размеру выборки:

$n \approx M / \delta^2$

max_noise among 65536 candidates $\sim \sqrt{2 \ln 65536 / n}$

Если одновременно ранжируются все 65536 кандидатов, то параметр $M = 10$ может оказаться слишком маленьким: порядок величины критического коэффициента здесь равен $2 \ln 65536 \approx 22.18$.

Поэтому после исправления выборки следующий полноценный прогон нужно делать заново и отдельно выбирать M , исходя уже не только из оценки одного bias, но и из ранжирования всех кандидатов сразу.

1.5.12. 5.12. Результат корректного прогона на 1000 ключах при $M = 1000$

После исправления выборки был выполнен новый прогон:

- каталог: results/key_recovery/20260507_224907_81499
- сводка: results/key_recovery/20260507_224907_81499/summary.csv
- итоговый отчёт: results/key_recovery/20260507_224907_81499/report.md

Параметры: - 1000 истинных ключей - seed = 1 - $M = 1000$ - top = 32 - threads = 10 - для каждой итерации использовались все 65536 открытых текстов

Главные результаты по качеству восстановления: - top-1: 940 / 1000 = 94.0% - top-3: 983 / 1000 = 98.3% - top-5: 987 / 1000 = 98.7% - top-10: 987 / 1000 = 98.7% - top-32: 995 / 1000 = 99.5% - top-100: 999 / 1000 = 99.9%

Ранги истинной пары ($K^{(1)}$, $K^{(2)}$): - медиана: 1 - средний ранг: 1.703 - минимум: 1 - максимум: 257

Дополнительная детализация по rank = 1: - 448 ключей: истинная пара была единственным лучшим кандидатом; - 492 ключа: истинная пара делила первое место с другими кандидатами; - всего 940 ключей: истинная пара была среди кандидатов с максимальным $|\delta_{\hat{t}}|$.

Статистика по $\delta^{(F_{\bar{K}})}$: - среднее: 0.0306868286 - медиана: 0.0299682617 - минимум: 0.0238037109 - максимум: 0.0509643555

Средние модули оценок: - mean $|TRUE_BIAS|$ = 0.0306868286 - mean $|BEST_BIAS|$ = 0.0307435913

Это означает, что после исправления выборки истинный кандидат почти совпадает с лучшим найденным по силе линейного сигнала.

Времена: - суммарно весь прогон: 1636.1944 сек, то есть около 27.27 минут - средняя итерация: 1.6362 сек - средний DELTA_TIME_SEC = 1.6113 сек - средний RECOVERY_TIME_SEC = 0.0242 сек

Сравнение со старым прогоном: - top-1: было 20.2%, стало 94.0% - top-32: было 34.7%, стало 99.5% - медианный ранг: было 128, стало 1 - средний ранг: было 3948.692, стало 1.703 - максимальный ранг: было 64950, стало 257

Вывод: - старый плохой результат был вызван ошибкой в методике отбора открытых текстов; - корректный прогон уже хорошо согласуется с теоретическим ожиданием: истинная пара почти всегда оказывается среди наиболее вероятных кандидатов. - этот прогон остаётся полезным как сильная контрольная точка, но следующий эксперимент по новому комментарию преподавателя нужно запускать уже при $M = 100$.

1.5.13. 5.13. Результат прогона на 1000 ключах при $M = 100$

- каталог: results/key_recovery/20260516_133452_83410
- сводка: results/key_recovery/20260516_133452_83410/summary.csv
- итоговый отчёт: results/key_recovery/20260516_133452_83410/report.md

Параметры: - 1000 истинных ключей - seed = 1 - $M = 100$ - top = 32 - threads = 10 - выборка открытых текстов: случайная без повторов из V_{16}

Так как $n = \text{ceil}(100 / \delta^{(2)})$, фактический размер выборки менялся: - среднее: 65206.946 - медиана: 65536 - минимум: 38501 - максимум: 65536

Главные результаты по качеству восстановления: - top-1: 939 / 1000 = 93.9% - top-3: 983 / 1000 = 98.3% - top-5: 987 / 1000 = 98.7% - top-10: 987 / 1000 = 98.7% - top-32: 995 / 1000 = 99.5% - top-100: 999 / 1000 = 99.9%

Ранги истинной пары ($K^{(1)}$, $K^{(2)}$): - медиана: 1 - средний ранг: 1.704 - минимум: 1 - максимум: 257

Дополнительная детализация по rank = 1: - 447 ключей: истинная пара была единственным лучшим кандидатом; - 492 ключа: истинная пара делила первое место с другими кандидатами; - всего 939 ключей: истинная пара была среди кандидатов с максимальным $|\delta_{\hat{t}}|$.

Средние модули оценок: - mean $|TRUE_BIAS|$ = 0.0306999375 - mean $|BEST_BIAS|$ = 0.0307579566

Это значит, что при $M = 100$ истинный кандидат по силе сигнала почти совпадает с лучшим найденным.

Времена: - суммарно весь прогон: 2067.3520 сек, то есть около 34.46 минут - средняя итерация: 2.0674 сек - средний DELTA_TIME_SEC = 2.0423 сек - средний RECOVERY_TIME_SEC = 0.0243 сек

Сравнение с прогоном при $M = 1000$: - top-1: было 94.0%, стало 93.9% - top-3: осталось 98.3% - top-32: осталось 99.5% - средний ранг: было 1.703, стало 1.704

Вывод: - эксперимент при $M = 100$, который просил преподаватель, подтверждает тот же качественный результат; - уменьшение M с 1000 до 100 практически не ухудшило качество восстановления; - истинная пара по-прежнему почти всегда оказывается среди наиболее вероятных кандидатов.